



## **COURS: LANGAGE C (PARTIE\_3)**

**Par: Dr. Mohamed Lemine Sidi**



## CONTENU DU COURS (PARTIE\_3)

- 8. Les Fonctions
- 9. Les structures de données composées
- 10. Les fichiers





## 8. LES FONCTIONS

### 8.1. Introduction

Une fonction est un bloc de code qui effectue une tâche spécifique. Elle permet de:

- Diviser un programme complexe en parties plus simples
- Réutiliser du code
- Améliorer la lisibilité et la maintenance
- Faciliter le débogage

Syntaxe :

```
type_de_retour nom_de_la_fonction(type_1 nom_param1, type_2 nom_param2, ...){  
    // Corps de la fonction: Les instructions à exécuter.  
    return valeur; // Optionnel selon le type de retour  
}
```

Exemple:

```
int addition(int a, int b) {  
    int resultat = a + b;  
    return resultat;  
}
```



## 8. LES FONCTIONS

### 8.1. Introduction (continue ...)

- **type\_de\_retour** : Spécifie le type de données que la fonction renvoie au code appelant. Si la fonction n'a pas besoin de renvoyer de valeur, on utilise le mot-clé void. Les types de retour courants incluent int, float, char, double, etc.
- **nom\_de\_la\_fonction** : C'est l'identificateur unique que vous utilisez pour appeler la fonction. Choisissez des noms descriptifs qui indiquent clairement ce que fait la fonction.
- **(type\_1 nom\_param1, type\_2 nom\_param2, ...)** : C'est la liste des paramètres de la fonction. Un paramètre est une variable qui reçoit une valeur lorsque la fonction est appelée. Chaque paramètre est spécifié par son type de données suivi de son nom. Une fonction peut ne pas avoir de paramètres (dans ce cas, les parenthèses sont vides ()).
- **return valeur;** : Cette instruction est utilisée pour renvoyer une valeur depuis la fonction vers le code appelant. Le type de valeur doit correspondre au **type\_de\_retour** déclaré dans l'en-tête de la fonction. Si le **type\_de\_retour** est **void**, l'instruction return; peut être utilisée pour quitter la fonction plus tôt que prévu, ou elle peut être omise.



## 8. LES FONCTIONS

### 8.2. Déclaration et Définition des fonctions

- **Déclaration:** On peut déclarer une fonction avant de la définir. La déclaration (ou prototype) indique au compilateur le nom de la fonction, son type de retour et les types de ses paramètres. Elle se termine par un point-virgule et ne contient pas le corps de la fonction. Exemple :

```
int addition(int, int); // prototype
```

- **Définition d'une fonction:** Elle contient le corps de la fonction. Il est obligatoire de définir une fonction pour pouvoir l'utiliser. Exemple:

```
int addition(int a, int b) {  
    return a + b;  
}
```

- **Appel d'une fonction:** Pour utiliser une fonction on doit l'appeler.

On appelle une fonction en utilisant son nom et en lui passant les arguments nécessaires. Exemple:

```
int resultat = addition(5, 3);
```



## 8. LES FONCTIONS

### 8.3. Types de Fonctions

8.3.1. Fonction avec retour et avec paramètres: C'est fonction qui exécute une tâche en utilisant des paramètres, et renvoie une valeur (typeX). Exemple:

```
int addition(int a, int b) {  
    return a + b;  
}
```

8.3.2. Fonction sans retour, avec paramètres: C'est fonction qui exécute une tâche en utilisant des paramètres, mais ne renvoie rien (void). Exemple:

```
void saluerPersonne(char nom[]) {  
    printf("Bonjour, %s !\n", nom);  
}
```

8.3.3. Fonction sans retour, sans paramètres: C'est fonction qui exécute une tâche autonome et ne renvoie rien (void). Exemple:

```
void saluer() {  
    printf("Bonjour!\n");  
}
```





## 8. LES FONCTIONS

8.3.4. Fonction avec retour, sans paramètres: C'est fonction qui exécute une tâche, et renvoie une valeur (typeX).

Exemple:

```
int obtenirNombreFixe() {  
    return 17;  
}
```





## 8. LES FONCTIONS

### 8.4. Portée des variables

- **Variables locales** : Les variables déclarées à l'intérieur d'une fonction (y compris les paramètres) ne sont accessibles qu'à l'intérieur de cette fonction. Elles sont créées lorsque la fonction est appelée et détruites lorsque la fonction se termine.
- **Variables globales** : Les variables déclarées en dehors de toute fonction sont appelées variables globales. Elles sont accessibles depuis n'importe quelle fonction du programme.

Exemple:

```
#include <stdio.h>
int variableGlobale = 10; // Variable globale
void afficherValeur() {
    int variableLocale = 5; // Variable locale à afficherValeur
    printf("Variable globale : %d\n", variableGlobale);
    printf("Variable locale : %d\n", variableLocale);
}
int main() {
    afficherValeur();
    return 0;
}
```





## 8. LES FONCTIONS

### 8.5. Passage d'arguments aux fonctions

Il existe principalement deux façons de passer des arguments à une fonction en C :

8.5.1. Passage par valeur: Une copie de la valeur de l'argument est passée à la fonction. Toute modification effectuée sur le paramètre à l'intérieur de la fonction n'affecte pas la variable d'origine dans le code appelant.

Exemple:

```
#include <stdio.h>
void incrementer(int x) {
    x = x + 1;
    printf("Valeur de x à l'intérieur de la fonction : %d\n", x);
}
int main() {
    int nombre = 10;
    incrementer(nombre);
    printf("Valeur de nombre après l'appel de la fonction : %d\n", nombre);
    return 0;
}
```



## 8. LES FONCTIONS

8.5.2. Passage par adresse: L'adresse mémoire de la variable est passée à la fonction. Cela permet à la fonction de modifier directement la valeur de la variable d'origine. On utilise des pointeurs (\*) pour déclarer les paramètres et l'opérateur d'adresse (&) pour passer l'adresse de la variable lors de l'appel de la fonction.

Exemple:

```
#include <stdio.h>
void incrementerParAdresse(int *x) {
    *x = *x + 1; // Utilisation de l'opérateur de déréférencement (*) pour accéder
                // à la valeur pointée par x
    printf("Valeur pointée par x à l'intérieur de la fonction : %d\n", *x);
}
int main() {
    int nombre = 10;
    incrementerParAdresse(&nombre); // Passage de l'adresse de la variable nombre
    printf("Valeur de nombre après l'appel de la fonction : %d\n", nombre);
    return 0;
}
```



## 8. LES FONCTIONS

### 8.6. Les fonctions récursives

Une fonction récursive est une fonction qui s'appelle elle-même au sein de sa propre définition.

#### 8.6.1. La structure d'une fonction récursive:

Toute fonction récursive bien conçue comporte généralement deux parties essentielles :

- Le cas de base (ou condition d'arrêt) : C'est la condition qui détermine quand la récursion doit s'arrêter. Sans un cas de base, la fonction s'appellerait indéfiniment, entraînant un débordement de pile (stack overflow), car chaque appel de fonction consomme de la mémoire sur la pile d'exécution. Le cas de base est crucial pour éviter une boucle infinie.
- L'étape récursive : C'est la partie où la fonction s'appelle elle-même, généralement avec un ensemble d'arguments modifié de manière à se rapprocher du cas de base. L'idée est de décomposer le problème initial en sous-problèmes plus petits et similaires.

Exemple : le calcul de la factorielle ....



## 8. LES FONCTIONS

### 8.6. Les fonctions récursives (continue ...)

Exemple : le calcul de la factorielle

```
#include <stdio.h>
int factorielle_recursive(int n) {
    // Cas de base : La factorielle de 0 est 1
    if (n == 0) {
        return 1;
    } else {
        // Étape récursive :  $n! = n * (n-1)!$ 
        return n * factorielle_recursive(n - 1);
    }
}

int main() {
    int nombre = 5;
    int resultat = factorielle_recursive(nombre);
    printf("La factorielle de %d est %d\n", nombre, resultat); // La factorielle de 5 est 120
    return 0;
}
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

### 9.1. Définition:

- Les structures de données composées permettent de regrouper plusieurs variables de types différents sous un même nom.
- Les principales structures composées sont les structures (**struct**), les unions (**union**) et les énumérations (**enum**).

### 9.2. Les structures (**struct**)

- Une structure permet de regrouper des variables de types différents.
- Elle est utile pour modéliser des objets complexes (ex: un étudiant, une voiture...).

Syntaxe:

```
struct NomDeStructure {  
    type1 champ1;  
    type2 champ2;  
    ...  
};
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

**Exemple:** Une structure Etudiant qui contient un nom (chaîne), un âge (int), et une moyenne (float).

```
#include <stdio.h>
struct Etudiant {
    char nom[50];
    int age;
    float moyenne;
};
main() {
    struct Etudiant e1 = {"Alice", 20, 15.5};
    printf("Nom : %s\n", e1.nom);
    printf("Âge : %d\n", e1.age);
    printf("Moyenne : %.2f\n", e1.moyenne);
}
```





## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

### 9.3. Structure avec typedef

- Le mot-clé typedef permet de créer un alias pour une structure. Cela rend son utilisation plus simple sans avoir à écrire struct à chaque fois.

**Syntaxe:**

```
typedef struct {  
    type1 champ1;  
    ...  
} NomAlias;
```

**Exemple:**

```
#include <stdio.h>  
typedef struct {  
    char nom[50];  
    int age;  
    float moyenne;  
} Etudiant;  
int main() {  
    Etudiant e1 = {"Mohamed", 22, 14.2};  
    printf("Nom : %s\n", e1.nom);  
    printf("Âge : %d\n", e1.age);  
    printf("Moyenne : %.2f\n", e1.moyenne);  
}
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

### 9.4. Structure imbriquée (structures dans des structures)

- Une structure imbriquée est une structure qui contient une autre structure comme un champ.

**Exemple:** La structure Personne contient une autre structure Adresse.

```
#include <stdio.h>

typedef struct {
    char ville[30];
    int codePostal;
} Adresse;

typedef struct {
    char nom[50];
    int age;
    Adresse adresse;
} Personne;
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

Exemple (continue ...)

```
main() {  
    Personne p = {"Jean", 30, {"Lyon", 69000}};  
    printf("Nom : %s\n", p.nom);  
    printf("Âge : %d\n", p.age);  
    printf("Ville : %s\n", p.adresse.ville);  
    printf("Code Postal : %d\n", p.adresse.codePostal);  
}
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

### 9.5. Tableaux de Structures

- Un tableau de structures permet de stocker plusieurs instances d'une même structure dans un tableau.
- C'est très utile pour gérer des collections d'objets similaires (ex : une liste d'étudiants).

Exemple:

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nom[50];
    int age;
    float moyenne;
} Etudiant;
```



## 9. LES STRUCTURES DE DONNÉES COMPOSÉES

Exemple (continue ...):

```
int main() {
    Etudiant classe[3]; // Saisie des données
    for (int i = 0; i < 3; i++) {
        printf("\nÉtudiant %d:\n", i + 1);
        printf("Nom: ");
        scanf("%s", classe[i].nom);
        printf("Age: ");
        scanf("%d", &classe[i].age);
        printf("Moyenne: ");
        scanf("%f", &classe[i].moyenne);
    } // Affichage des données
    printf("\nListe des étudiants:\n");
    for (int i = 0; i < 3; i++) {
        printf("\nÉtudiant %d:\n", i + 1);
        printf("Nom: %s\n", classe[i].nom);
        printf("Age: %d\n", classe[i].age);
        printf("Moyenne: %.2f\n", classe[i].moyenne);
    }
}
```